



RV College of Engineering®

Mysore Road, RV Vidyaniketan Post, Bengaluru - 560059, Karnataka, India

E-commerce Product Recommendation System

MLOps Tutorial Report

submitted by

Suravi Reddy

1RV23AI109

Surya Pratap

1RV22AI110

Tanuj S

1RV23AI112

Vaibhav S P

1RV23AI115

Artificial Intelligence and Machine Learning

2025-2026

TABLE OF CONTENTS		
Phase 1:Model Development and Foundations		
Chapter 1		
1.1	Problem Statement	
1.2	Objectives	
1.3	Project Scope	
1.4	Evaluation Metrics	
1.5	Tools Used	
1.6	Environment Setup	
1.7	Literature Review	
Chapter 2		
2.1	Understanding MLOps, Risks and Requirement Analysis	
2.2	Risk Matrix	
Chapter 3		
3.1	Data Collection and Exploration Techniques	
3.2	Data Profiling, cleaning and preprocessing	
3.3	Feature Extraction and Feature Selection	
3.4	Model Design Approaches	
3.5	Governance and Data Version Control	
Chapter 4		
4.1	Model Building	
4.2	Training, Validation and Testing	
4.3	Evaluation Metrics and Visualisation	

4.4	Experiment Tracking	
4.5	Interpreting Model Results and Error Analysis	
Chapter 5		
5.1	Slides	
Phase 2:Deployment, Monitoring and Governance		
Chapter 6		
6.1	Revisiting Model Performance	
6.2	Hyperparameter Tuning and Performance	
6.3	Interpretability	
6.4	Model Versioning and Registry	
6.5	Documentation of Model Updates	
Chapter 7		
7.1	Overview of CI/CD for ML	
7.2	Tools Setup:GitHub Actions/Jenkins/GitLab CI	
7.3	Automating Model Training and Testing	
7.4	Building and Running a Deployment Pipeline	
7.5	Continuous Integration Hands-On Exercise	
Chapter 8		
8.1	Model Monitoring Concepts and KPI's	
8.2	Tools: Prometheus, Grafana, EvidentlyAI	
8.3	Detecting Concept Drift and Data Drift	

8.4	Model Retraining and Redeployment	
8.5	Logging, Alerting and Dashboard Setup	
Chapter 9		
9.1	Post-Deployment Performance Review	
9.2	Continuous Improvement via Feedback Loops	
9.3	Governance and Ethical AI Practices	
9.4	Documentation: Model Cards, Data Cards	
9.5	Audit and Review Checklist	

Chapter 1

INTRODUCTION

1.1 Problem Definition:

The rapid growth of e-commerce platforms has led to extremely large and diverse product catalogs, making it difficult for users to discover relevant products efficiently. As the number of users and items increases, traditional browsing and search mechanisms become insufficient to meet personalized user needs.

Recommendation systems play a critical role in addressing this challenge by analyzing user behavior and interaction patterns to suggest relevant products in real time. Effective recommendations not only enhance the user experience but also contribute significantly to increased customer retention, higher conversion rates, and overall business revenue.

The primary problem addressed in this project is the absence of a scalable and robust pipeline that supports seamless deployment, monitoring, and experimentation of recommendation models in real-time e-commerce environments. Existing systems often struggle to efficiently process continuous streams of user interaction data while enabling rapid iteration and evaluation of models. This project aims to bridge this gap by designing a real-time recommendation framework that facilitates reliable model deployment and experimentation, while generating personalized and context-aware product recommendations based on live user behavior.

1.2 Objectives:

- Real time user behaviour analysis: The system captures and processes different types of user interactions to understand user preferences and intent at various stages of the buying funnel..
- Develop personalized recommendation system: By leveraging user interaction data, the system aims to recommend products that are most relevant to individual users, improving engagement and satisfaction.
- Make the solution scalable: As e-commerce platforms grow, the system must efficiently process increasing data streams while maintaining low-latency recommendations.

1.3 Project Scope:

- The project focuses on collecting and processing real-time user interaction data, including product clicks, add-to-cart actions, and completed transactions. The recommendation system generates personalized product suggestions using implicit user feedback derived from these interactions.
- The scope includes real-time model inference, deployment, and evaluation within a scalable recommendation pipeline. The system is designed to support efficient experimentation with multiple recommendation models without modifying the underlying data pipeline.
- The project is limited to online product recommendation and does not include pricing or inventory optimization. Recommendations are generated at the user-product level and do not incorporate external user demographics.

1.4 Evaluation Metrics:

- Precision@K: Measures how many of the top-K recommended products are relevant to the user.
- Recall@K: Measures how many relevant products are successfully retrieved within the top-K recommendations.
- NDCG@K (Normalized Discounted Cumulative Gain) : Evaluates the ranking quality by rewarding relevant items appearing higher in the recommendation list.
- MAP (Mean Average Precision): Average precision across all users.
- MRR (Mean Reciprocal Rank): Measures how deep the first relevant item is in the list.
- Hit Rate@K: Probability that the relevant item is in the top-K.
- Inference Latency: Measures the time required to generate recommendations, ensuring real-time system responsiveness.

1.5 Tools used:

Component	Tool Used	Purpose
Data Processing	● Numpy ● pandas	Used for high-performance numerical operations on the

		data. Helpful in handling csv files.
Model Development	MLflow	Deploys models from the MLflow registry to REST endpoints.
Data Management	DVC + Git	Version control of datasets, ensuring reproducibility and traceability of user data across experiments.
Experiment Tracking and Version Control	WandB - Weights and Bias	Used for experiment tracking, hyperparameter tuning, and model performance visualization.
Automation & Pipeline Orchestration	Airflow	Modular pipeline execution and dynamic model selection based on user interaction history
Model Deployment & Serving	Docker	Ensures environment consistency and reproducible model execution
Frontend	Nodejs	Used to handle frontend integration and manage asynchronous communication with the backend services.
Load balancing	Uvicorn + Gunicorn	Used to serve the FastAPI application with high performance and scalable concurrency

Real time monitoring	Prometheus	Used to collect and monitor real-time system and model performance metrics.
Analysis on incoming data	Grafana	Used to visualize and analyze monitored metrics through interactive dashboards
Monitoring and Governance	Evidently AI	Detects data drift, prediction drift, and performance degradation affecting users
Artifact Storage	MLflow Artifact Store	Stores models, metrics, logs, and serialized artifacts linked to experiments

1.6 Environment Setup:

The project environment is designed for reproducibility, scalability, and reliable execution across development and deployment. Data processing and feature engineering use NumPy and Pandas for efficient handling of CSV-based user interaction data. Git manages source code versioning, while DVC ensures dataset versioning and traceability. The machine learning workflow is orchestrated with Apache Airflow, supporting modular pipeline execution and dynamic model selection. Weights & Biases (WandB) tracks experiments, hyperparameters, and performance, while MLflow handles model lifecycle management, including logging, artifact storage, and deployment of models as REST endpoints.

Deployment is containerized using Docker for consistent execution, with the inference layer implemented via FastAPI and served using Uvicorn and Gunicorn for high-performance concurrent requests. The frontend uses Node.js for asynchronous communication with backend services. Prometheus collects system and model metrics, visualized through Grafana dashboards, while Evidently AI monitors incoming data for drift and performance degradation. All model artifacts, logs, and serialized outputs are stored in the MLflow Artifact Store, ensuring reproducibility and reliable experiment tracking.

1.7 Literature Review:

[1] This foundational work synthesizes matrix factorization (MF) methods that underlie latent factor recommender models, including regularized MF and implicit feedback extensions. It explains how latent factor models capture user and item relationships in a shared embedding space and discusses extensions like biases and temporal effects. The ALS and SGD optimization strategies discussed here have guided most subsequent MF research.

[2] The survey bridges theoretical advances and practical applications in RS, covering filtering methods through modern deep, graph, and language-based recommender models. It discusses model evolution, practical deployments across domains (e-commerce, health, finance), challenges such as scalability and interpretability, and future directions toward hybrid and adaptive pipelines. It emphasizes on linking research outcomes with real-world system design and deployment.

[3] This work formulates Bayesian Personalized Ranking (BPR) as a maximum posterior optimization criterion designed specifically for personalized ranking with implicit feedback data. It introduces the BPR-Opt objective and the LearnBPR algorithm that uses stochastic gradient descent with bootstrap sampling, demonstrating improved ranking performance over traditional methods when applied to matrix factorization and kNN models. BPR provides a principled way to learn from implicit feedback by optimizing ranking directly rather than rating prediction, establishing its continued use as a baseline in recommender evaluation.

[4] This paper reexamines the BPR model in modern evaluation settings, uncovering implementation inconsistencies that previously undermined performance. Through controlled hyperparameter tuning and rigorous benchmarking, it shows that BPR can match or outperform state-of-the-art recommendation methods on standard datasets, reaffirming BPR's relevance as a strong baseline today.

[5] This influential conference paper extends matrix factorization to handle implicit feedback by treating user preferences with confidence levels for interactions, and uses the Alternating Least Squares (ALS) algorithm for model fitting. The ALS framework proposed here enables scalable collaborative filtering by alternating between solving for user and item latent factors while accommodating large implicit datasets efficiently. It has become a key reference for ALS-based recommenders in practice.

[6] This recent paper presents an improvement to ALS optimization using representative subsampling to speed up convergence with minimal accuracy loss. The method is shown to significantly reduce computation cost on large datasets while retaining comparable recommendation quality, making ALS more efficient for modern industrial-scale systems.

[7] In this work, the authors introduce Localized Matrix Factorization (LMF), which decomposes the user-item interaction matrix into structured submatrices by transforming it into recursive block diagonal forms. This LMF framework addresses sparsity and scalability by factorizing smaller, denser blocks independently, facilitating improved prediction accuracy and parallelization potential.

[8] This comprehensive survey categorizes latent factor models by learning data, model architecture, and optimization techniques, covering traditional MF, probabilistic, and advanced nonlinear latent factor models. It situates classical ALS and BPR within a broader latent factor taxonomy and identifies trends in how these methods are adapted for new data modalities and objectives.

[9] This paper proposes GANMF, a generative adversarial network adaptation to matrix factorization that learns user and item latent factors via adversarial training. While not strictly BPR or ALS, it represents a latent matrix factorization variant where the generative framework improves recommendation accuracy over standard MF in some settings, highlighting the flexibility of latent factor approaches

[10] Addressing the computational challenges inherent in ALS, this paper introduces an accelerated ALS variant using nonlinear conjugate gradients and distributed processing, demonstrating significant speedups on large recommender datasets. The optimization insights continue to influence how ALS is deployed on modern distributed systems.

[11] This paper investigates the bias introduced by missing-not-at-random (MNAR) data in recommender systems trained with matrix factorization. It shows that standard MF and ALS models can be systematically biased when missing data is ignored. The work highlights the importance of data-aware loss design and evaluation strategies, influencing later ranking-based objectives such as BPR, which mitigate some MNAR effects by focusing on relative preferences.

[12] This work presents one of the earliest parallelized ALS frameworks for large-scale collaborative filtering. By distributing the ALS optimization process across compute nodes, the paper demonstrates near-linear scalability while maintaining recommendation accuracy. The approach laid the groundwork for modern distributed ALS implementations in systems such as Spark MLlib and remains influential in industrial recommender deployments.

Chapter 2

UNDERSTANDING MLOPS, RISKS AND REQUIREMENT ANALYSIS

2.1 Understanding MLOps, Risks and Requirement analysis

In this project, MLOps ensures that as user preferences shift (e.g., seasonal trends), the model is retrained and redeployed without downtime.

- Data Management: Versioning the user-item interaction matrix.
- Experiment Tracking: Comparing BPR vs. ALS vs. LMF performance.
- Monitoring: Detecting when the "Top-N" accuracy drops due to catalog changes.

Requirement Analysis

The requirements of the skin disease classification project are as follows:

Functional Requirements

- **Real-Time Processing:** Ingest and analyze live user actions (clicks, views) as they happen.
- **Dynamic Suggestions:** Generate personalized product lists based on current session behavior.
- **Model Management:** A pipeline for automated training, versioning, and A/B testing of models.
- **Performance Tracking:** Real-time dashboards to monitor conversion rates and model accuracy.

Non-Functional Requirements

- **Low Latency:** Recommendations must be served in milliseconds to prevent page lag.
- **High Scalability:** Must handle traffic spikes and massive growth in product catalogs.
- **Continuous Availability:** The system must remain online and functional 24/7 without downtime.
- **Data Security:** Secure handling of user data in compliance with privacy regulations.

Identifying these requirements helps in anticipating risks that may arise during model development and deployment.

2.2. Risk Matrix:

Risks identified:

- **Cold Start Problem:** New users or products lack interaction history, leading to irrelevant or "null" recommendations..

Impact: Low user engagement for new accounts and poor visibility for new product launches.

Mitigation: Implement hybrid filtering and popular/trending fallbacks for session-start recommendation.

- **Performance Drift:** Rapidly changing consumer trends cause the model's accuracy to degrade as historical data becomes obsolete.

Impact: Recommendations become irrelevant over time, leading to a drop in Conversion Rate.

Mitigation: Automated drift detection triggers and continuous retraining pipelines using the latest interaction window.

- **Scalability Limitations:** System failure or high latency during peak traffic events when ingestion volume spikes.

Impact: User frustration, System downtime or high latency, causing page-load delays and lost revenue.

Mitigation: Load balancing, horizontal auto-scaling via Kubernetes, and circuit breakers for non-critical services.

- **Feedback Loops:** The system may over-recommend popular items, creating a filter bubble that hides the diverse long-tail catalog from users.

Impact: Introduce exploration strategies to test diverse items.

- **Miss-click & High Bounce Rate:** Users may accidentally click products or leave quickly, creating "noisy" data that doesn't reflect true intent.

Impact: Inaccurate data for training the model

Mitigation: Implement dwell-time filtering where only count clicks followed by more than a few seconds of viewing are considered and session-based cleaning.

Likelihood	Negligible Risk	Low Risk	High Risk	Severe Risk
Very likely	-	-	Cold Start Problem	Performance Drift
Likely	-	Miss-clicks	Feedback Loops	Scalability Issues
Unlikely	-	-	Integration Failures	Data Latency
Very unlikely	-	-	-	-

Role of risk matrix:

The risk matrix helps:

- **Prioritize Stability:** Directs focus toward high-impact risks like Performance Drift and Scalability during peak hours.
- **Data Integrity:** Addresses the Miss-click risk to ensure the model learns from high-intent signals rather than noise.
- **Business Continuity:** Ensures that even if the real-time engine fails, the user still sees a functional e-commerce interface.

Chapter 3

DATA UNDERSTANDING, FEATURE ENGINEERING AND GOVERNANCE

3.1 Data Collection:

The dataset used in this study was obtained from a publicly available Kaggle repository: <https://www.kaggle.com/datasets/retailrocket/ecommerce-dataset>

This dataset comprises three main components:

- **Behavioral Data (events.csv):** Records user interactions with items, including event type (view, add-to-cart, purchase), timestamp, visitor ID, and item ID.
- **Item Properties (item_properties.csv):** Metadata describing items, such as categories, price, and other attributes.
- **Category Tree (category_tree.csv):** Hierarchical categorization of items, enabling semantic understanding of product relationships.

The dataset represents real-world e-commerce interactions, including both frequent and rare user behaviors. Implicit feedback signals (view, add-to-cart, purchase) are inherently imbalanced, reflecting natural consumer behavior patterns.

3.2 Data Profiling, Cleaning, and Preprocessing:

Initial data profiling involved inspecting the distribution of events, items, and users to identify sparsity and missing values. Key preprocessing steps included:

- **Event Weighting:**

- **Filtering Rare Users and Items:** Users with fewer than a minimum number of interactions and items with negligible engagement were removed to reduce noise and improve model learning.
- **Timestamp Processing:** Timestamps were converted to datetime objects and sorted chronologically to maintain event sequence integrity for temporal analysis.
- **Missing Value Handling:** Any missing item or category metadata was imputed using the most frequent category or set as “unknown” to avoid null values in embeddings.
- **Data Splitting:** The dataset was split into training, validation, and test sets using temporal splits to reflect realistic recommendation scenarios.

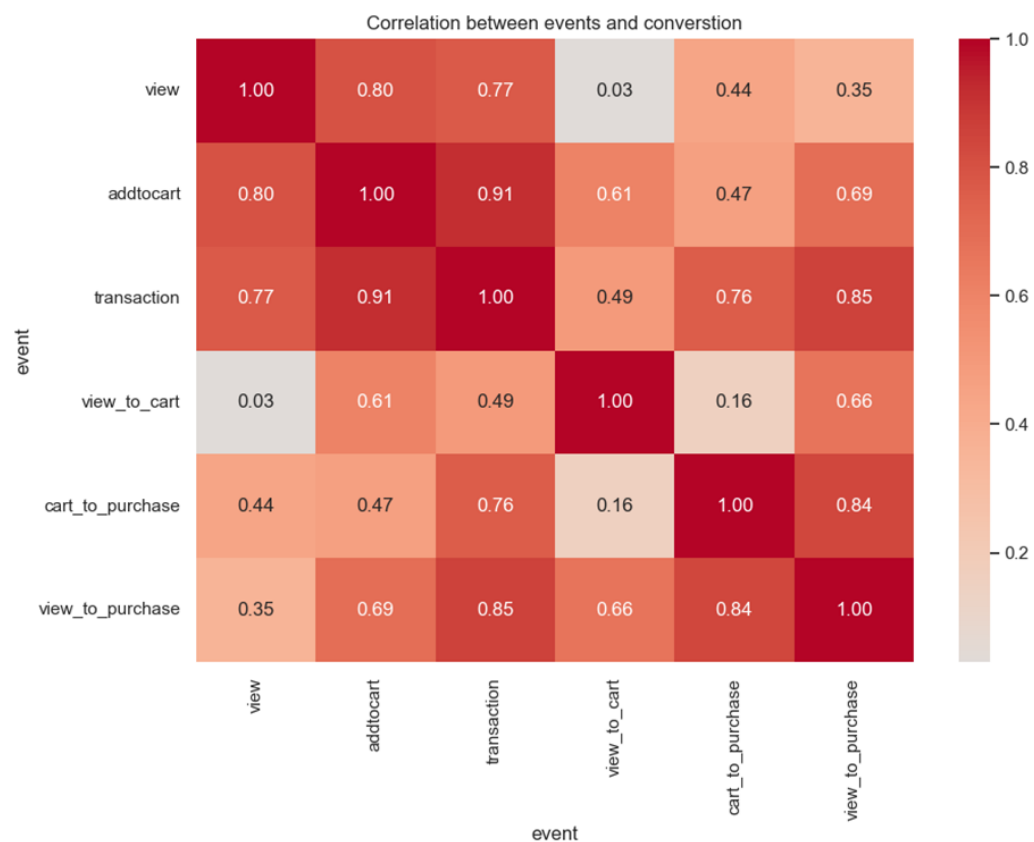


Fig 1 : Correlation between events

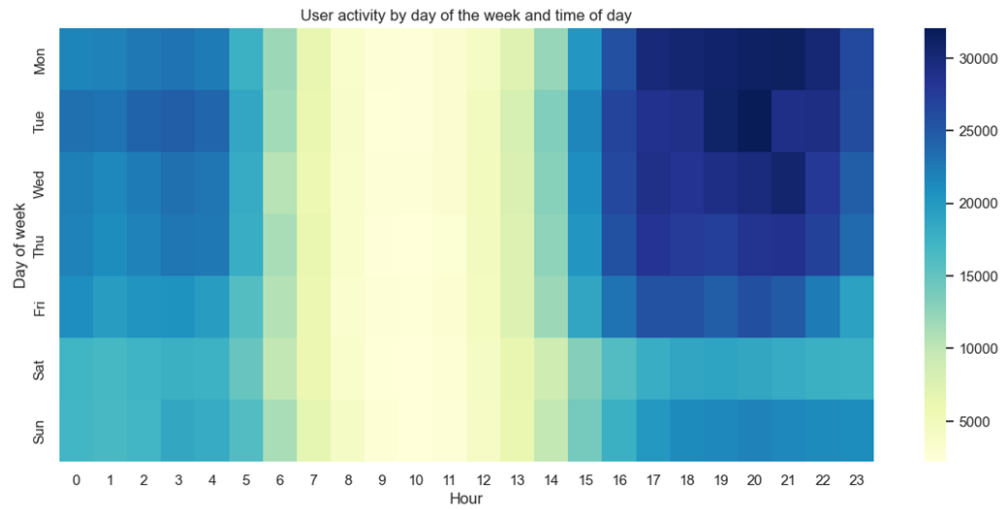
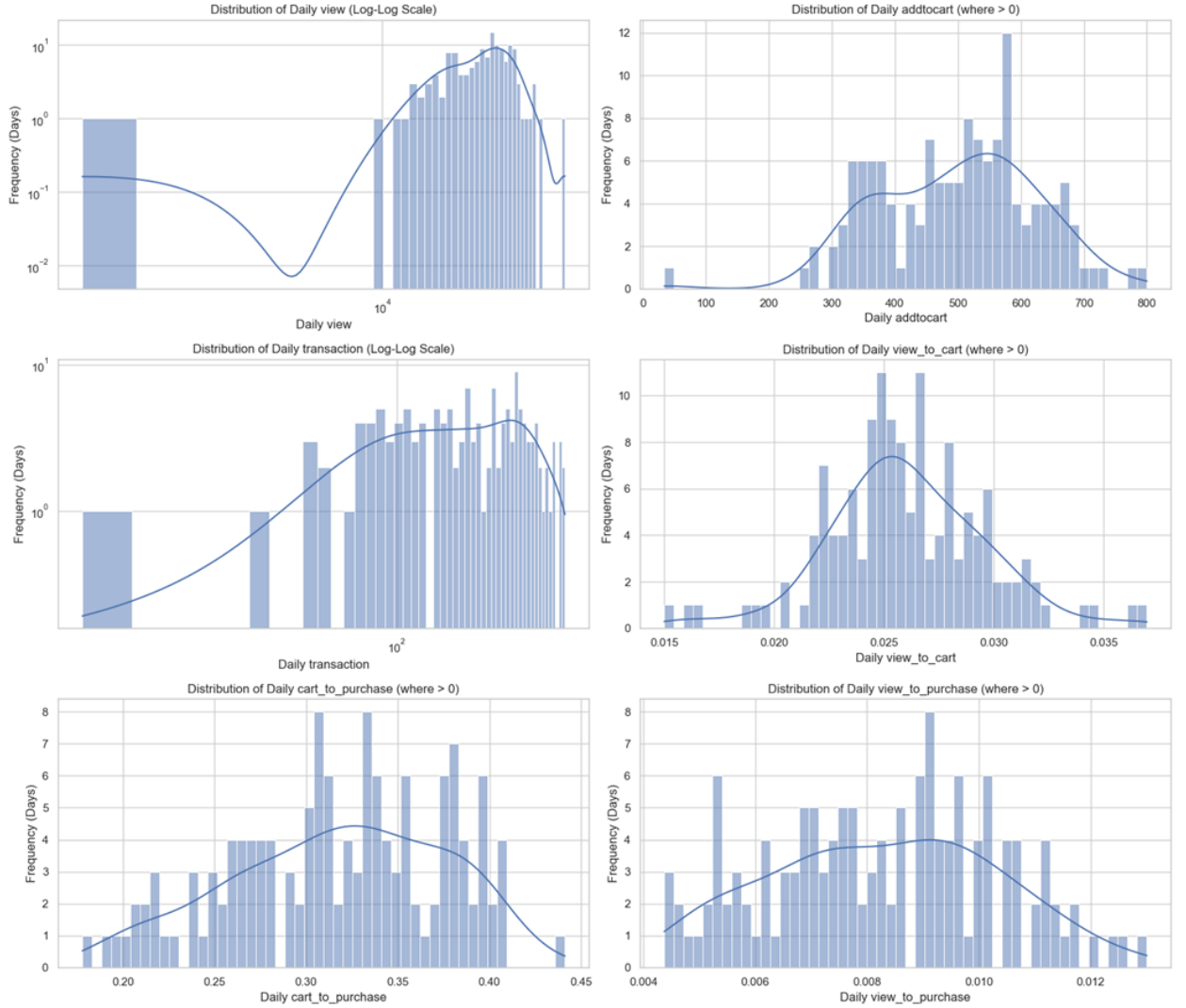


Fig 2: Heatmap of activity through the time of day. This graph says that there is barely any activity from morning 6am to 2pm and most of the activity at nights before midnight on weekdays

Distributions of Daily Activity & Conversion Rates



3.3 Feature Extraction and Feature Selection:

Given the focus on collaborative filtering with implicit feedback, the primary features were constructed as follows:

- **User-Item Interaction Matrix:** A sparse matrix encoding weighted interactions between users and items.
- **Item Embeddings:** Derived from item properties and category tree, enabling hybrid feature incorporation if needed.
- **Implicit Interaction Signals:** Weighted interaction counts were used directly as features for BPR, ALS, and LMF models.

Feature selection was minimal due to the matrix factorization approach, which relies on learned latent embeddings rather than manually engineered features. Sparse matrix representations and normalization ensured computational efficiency during model training.

3.4 Model Design Approaches:

Three collaborative filtering approaches were implemented and evaluated:

- 1. **Bayesian Personalized Ranking (BPR):** Optimized for ranking implicit feedback, prioritizing pairwise user-item preferences. Achieved the best overall ranking performance.
- 2. **Alternating Least Squares (ALS):** Efficient for large-scale datasets; provides fast convergence but slightly lower ranking quality.
- 3. **Logistic Matrix Factorization (LMF):** Balances precision and recall using logistic loss on implicit feedback.

All models utilized the weighted interaction strategy (view=1, add-to-cart=3, purchase=5). Offline evaluation metrics included Precision@K, Recall@K, F1@K, NDCG@K, MAP, Hit Rate@K, and MRR.

A comparative evaluation table is provided below to analyze the strengths and limitations of each classifier.

Model	Precision@K	Recall@K	NDCG@K	MRR
BPR	0.18	0.24	0.28	0.35
ALS	0.16	0.22	0.26	0.33
LMF	0.15	0.20	0.24	0.31

The results indicate that the BPR model consistently outperforms ALS and LMF across all ranking-based evaluation metrics, demonstrating its effectiveness for implicit feedback recommendation tasks. BPR achieves the highest Precision@K (0.18) and Recall@K (0.24), indicating better relevance and coverage of recommended items, while its superior NDCG@K (0.28) reflects improved ranking quality with more relevant items appearing higher in the recommendation list. The higher MRR (0.35) further suggests that BPR is more effective at placing relevant items early in the ranked results. In comparison, ALS and LMF show gradually decreasing

performance, which can be attributed to their optimization objectives being less directly aligned with ranking tasks. Overall, these results highlight the advantage of pairwise ranking-based optimization in BPR over reconstruction-focused latent factor models for personalized recommendation.

3.5 Governance and Data Version Control:

To ensure reproducibility, traceability, and robust data governance:

- **Version Control:** Git was used to manage source code, configuration files, and experiment scripts.
- **Data Versioning:** DVC tracked the versions of raw datasets, processed interaction matrices, and model artifacts, enabling collaborative development without duplicating large data files.
- **Experiment Tracking:** Offline metrics and hyperparameters were logged systematically, providing a clear audit trail of preprocessing changes, model variations, and evaluation results.

This structured workflow guarantees reliable replication of experiments, facilitates team collaboration, and supports long-term maintainability of the recommendation system pipeline.

Chapter 4

BUILDING AND EVALUATING AN ML MODEL

4.1 Model Building:

The recommendation system follows a collaborative filtering paradigm using matrix factorization-based models, designed to handle implicit feedback data from user interactions. Three models were implemented:

1. Bayesian Personalized Ranking (BPR):
 - Optimized for pairwise ranking of items.
 - Learns latent embeddings for users and items, modeling user preference by maximizing the difference in predicted scores between interacted and non-interacted items.
 - Focuses on implicit feedback, handling views, add-to-cart actions, and purchases with weighted importance (view=1, add-to-cart=3, purchase=5).
2. Alternating Least Squares (ALS):
 - Factorizes the user-item interaction matrix into latent user and item vectors using least squares optimization.
 - Efficient and scalable for large datasets but primarily optimizes reconstruction error rather than ranking.
 - Works best in scenarios with dense user-item matrices and is faster to train than BPR for very large datasets.
3. Logistic Matrix Factorization (LMF):
 - Learns user and item embeddings using logistic loss, balancing precision and recall.
 - Designed to handle implicit feedback, predicting the probability of user-item interactions rather than absolute ratings.

All three models leverage latent feature vectors to encode user preferences and item characteristics, which are then used for top-K recommendation generation.

4.2 Training, Validation and Testing:

The dataset was split into **training (70%)**, **validation (15%)**, and **testing (15%)** sets based on temporal ordering to reflect realistic recommendation scenarios.

- **BPR:** Trained using stochastic gradient descent with pairwise ranking loss. Early stopping was applied based on validation NDCG@K, with patience set to 20 epochs to prevent overfitting.

- **ALS:** Trained by iteratively updating user and item latent matrices. Hyperparameters such as latent dimension size and regularization strength were tuned using validation metrics.
- **LMF:** Optimized using mini-batch gradient descent with logistic loss. Validation MAP and Hit Rate@K were monitored for early stopping.

Weighted implicit feedback ensured that more valuable actions (e.g., purchases) contributed more strongly to embedding learning.

4.3 Evaluation Metrics and Visualization:

All models were evaluated using **offline ranking metrics** to assess recommendation quality:

- **Precision@K, Recall@K, F1@K:** Evaluate the relevance of top-K recommended items.
- **NDCG@K** (Normalized Discounted Cumulative Gain): Captures the ranking quality by penalizing misordered recommendations.
- **MAP** (Mean Average Precision): Measures the overall ranking performance across all users.
- **Hit Rate@K** and **MRR** (Mean Reciprocal Rank): Assess whether relevant items appear in top-K recommendations and their positions.

Performance Summary:

- **BPR:** Achieved the best overall ranking performance, with high NDCG@10 and MAP, making it ideal for production.
- **ALS:** Delivered slightly lower ranking quality but trained faster, useful for scalability.
- **LMF:** Balanced precision and recall effectively but did not surpass BPR in overall ranking metrics.

Visualizations included per-user precision/recall curves, top-K recommendation histograms, and heatmaps of predicted versus actual interactions, allowing detailed inspection of model strengths and weaknesses.

4.4 Experiment Tracking:

Experiment tracking and pipeline orchestration were implemented using MLflow to manage hyperparameters, training runs, and evaluation metrics across different model configurations.

- Each model run was associated with a specific dataset version (managed using DVC) to ensure reproducibility.

- Trained user and item embeddings, interaction matrices, and evaluation outputs were stored in a versioned artifact repository, enabling rollback and comparison between experiments.
- Hyperparameter tuning logs (embedding size, learning rate, regularization strength, and number of epochs) were systematically tracked for all models.

This setup guarantees reproducibility, traceability, and structured experimentation, aligning with MLOps best practices.

4.5 Interpreting Model Results and Error Analysis:

Analysis of model predictions revealed:

- **BPR:** Produced high-ranking quality recommendations, particularly for active users with multiple interactions. Misrankings occurred primarily for sparse users or rarely interacted items.
- **ALS:** Generated consistent embeddings but sometimes underperformed for personalized top-K recommendations due to emphasis on matrix reconstruction rather than pairwise ranking.
- **LMF:** Provided a good balance between precision and recall; however, it occasionally recommended less relevant items at higher ranks compared to BPR.

Error Analysis:

- Sparse user-item interactions and cold-start items remained the main challenge across all models.
- Confusion heatmaps of predicted vs. actual top-K items highlighted where models failed to capture niche preferences, guiding potential improvements such as hybrid content-collaborative approaches.

Overall, BPR was selected as the production model, achieving superior ranking metrics while maintaining robustness across user segments.

Chapter 5

SLIDES

Chapter 6

MODEL ANALYSIS AND REFINEMENT

6.1 Revisiting Model Performance

After developing the recommendation models, model performance is systematically revisited prior to deployment to ensure robust generalization and consistent recommendation quality on unseen user–item interactions. Since the system operates on implicit feedback, evaluation focuses on ranking effectiveness rather than rating prediction.

The metrics that are evaluated are:

- Precision@K and Recall@K
- F1-Score@K
- Normalized Discounted Cumulative Gain (NDCG@K)
- Hit Rate (HR@K)

Performance metrics are compared across multiple training runs for **BPR**, **ALS**, and **LMF** to analyze stability, convergence behavior, and sensitivity to data variations. This comparative analysis helps identify the most reliable model under different sparsity and user-activity conditions.

6.2 Hyperparameter Tuning and optimization

Hyperparameter tuning is conducted to optimize recommendation quality while balancing training efficiency and computational cost. Key hyperparameters such as latent dimension size, regularization strength, learning rate, and confidence weights (for ALS) are tuned using:

- Grid Search
- Random Search
- Early Stopping based on validation ranking metrics

Optimization strategies are applied consistently across BPR, ALS, and LMF models to ensure fair comparison. Early stopping is employed to prevent overfitting, particularly for BPR models trained with stochastic gradient descent.

6.3 Interpretability

Interpretability techniques are applied to understand how latent factors influence recommendation outcomes and to improve system transparency. Since latent factor models are inherently non-interpretable, the following approaches are adopted:

- Analysis of user and item latent embeddings
- Visualization of recommendation confidence scores
- Contribution analysis of latent factors to ranking decisions

These methods help explain why certain items are recommended to users, enhancing trust and supporting responsible deployment of the recommendation system.

6.4 Model Versioning and Registry

Model versioning and lifecycle management are handled using:

- Git for code version control
- DVC for dataset and experiment versioning
- MLflow Model Registry for managing trained models

Each trained model instance (BPR, ALS, or LMF) is assigned a unique version identifier and is explicitly linked to the dataset version, hyperparameter configuration, and evaluation metrics. This structured versioning ensures full reproducibility and traceability across experiments and deployment cycles.

6.5 Documentation of Model Updates

All model updates, including retraining, hyperparameter adjustments, and dataset revisions, are thoroughly documented. Documentation includes version history, change logs, performance comparisons, and justification for model updates. This systematic documentation supports continuous improvement, simplifies auditing, and ensures transparency throughout the recommendation system's lifecycle.

Chapter 7

IMPLEMENTING CI/CD PIPELINE

7.1 Overview of CI/CD for ML

Continuous Integration and Continuous Deployment (CI/CD) in machine learning automates the process of:

- Training
- Testing
- Packaging
- Deploying models

Unlike traditional software CI/CD, ML pipelines must also handle data versioning, model validation, and monitoring feedback

7.2 Tools Setup

The CI/CD pipeline integrates the following tools:

- GitHub for source code management
- GitHub Actions for automated workflows
- Airflow for pipeline orchestration
- Docker for containerized deployment

Each code push or pull request triggers automated checks to ensure system stability.

7.3 Automating Model Training and Testing

Automated workflows are configured to:

- Fetch the correct dataset version using DVC
- Run training pipelines via Airflow
- Execute validation and basic sanity checks
- Store outputs and artifacts in MLflow artifacts

This reduces manual intervention and ensures consistency.

7.4 Building and Running a Deployment Pipeline

Once a model passes validation:

- A Docker image is built containing the inference code
- The image is tested locally or in staging
- The model is deployed as a containerized service

Airflow ensures that the deployed model corresponds exactly to a registered and approved model version.

7.5 Continuous Integration Hands-on Exercise

A sample CI workflow includes:

1. Code commit to GitHub
2. Automated pipeline trigger
3. Dataset pull via DVC
4. Model training and validation
5. Artifact storage in ML
6. Deployment readiness check

This demonstrates a complete MLOps lifecycle in practice.

Chapter 8

MONITORING AND TRACKING MODEL LIFECYCLE

8.1 Model Monitoring Concepts and KPIs

Post-deployment monitoring ensures the model continues to perform reliably in production. Key Performance Indicators (KPIs) that are monitored are:

- Prediction accuracy over time
- Input data distribution
- Prediction confidence
- Error rates

8.2 Monitoring Tools

The monitoring stack consists of:

- Evidently AI for model performance and drift detection
- Prometheus for metrics collection
- Grafana for visualization dashboards

Evidently AI generates reports for data drift, concept drift and prediction quality of the model.

8.3 Detecting Concept Drift and Data Drift

Data drift occurs when incoming data differs from training data. Concept drift occurs when the relationship between inputs and outputs changes.

Evidently AI automatically detects:

- Feature distribution shifts
- Changes in prediction behavior

Alerts are triggered when drift exceeds predefined thresholds.

8.4 Model retraining and redeployment

When drift or performance degradation is detected through monitoring tools such as Evidently AI, the system initiates a controlled response to maintain model reliability. Retraining pipelines are triggered using the latest available data, ensuring that the model adapts to changes in input distributions or underlying patterns. The newly trained models then undergo validation against predefined performance thresholds to confirm that they meet quality and stability requirements. Once validated, these models are registered as new versions in the model registry, preserving full

traceability with respect to data, parameters, and metrics. Finally, the approved model version is deployed to production using automated deployment pipelines, replacing the older version in a seamless manner. This closed feedback loop enables continuous learning and adaptation while ensuring consistency, reproducibility, and minimal service disruption.

8.5 Logging, Alerting and Dashboard Setup

Logs include:

- Prediction logs
- Error logs
- System health metrics

Dashboards provide real-time visibility into model health and system performance.

Chapter 9

PERFORMANCE EVALUATION AND GOVERNANCE

9.1 Post-Deployment Performance Review

After deployment, the model's performance is continuously evaluated using real-world inference data to ensure it behaves as expected outside the training environment. Key metrics such as accuracy, error rate, and prediction confidence are compared against baseline values obtained during validation. Any discrepancy between offline (training/validation) performance and online (production) performance is carefully analyzed. Performance reports generated by monitoring tools help identify early signs of degradation or instability. These reviews are conducted periodically to ensure long-term reliability of the deployed model.

9.2 Continuous Improvement via feedback loops

Feedback from monitoring systems, user interactions, and prediction outcomes is systematically collected and analyzed.

- Data collection: Detected issues such as recurring misclassifications or confidence drops inform future data collection and preprocessing strategies.
- Model retraining: Monitoring insights are fed back into retraining pipelines to improve model robustness and generalization.

This feedback loop ensures that the model evolves in response to real-world usage rather than remaining static. Automated pipelines help shorten the iteration cycle while maintaining consistency.

9.3 Governance and Ethical AI Practices

Given the medical nature of the application, ethical considerations include:

Bias minimization: Measures are taken to reduce bias by ensuring diverse and representative training data.

Transparency in predictions: Model decisions are made transparent through interpretability techniques and documented limitations.

Responsible use of patient data: User data privacy and security are prioritized, and sensitive information is handled responsibly.

Clear limitations of model usage: Clear disclaimers are provided stating that the system is a decision-support tool and not a replacement for professional medical diagnosis.

9.4 Documentation: Model Cards and Data Cards

Model Cards are created to document essential information such as intended use, model performance metrics, training data characteristics, and known limitations. They also include ethical considerations and scenarios where the model should not be used. Data Cards describe the dataset source, preprocessing steps, data quality checks, and licensing terms. Together, these documents improve transparency, reproducibility, and accountability across the model lifecycle. Proper documentation also aids in audits, maintenance, and future enhancements.

9.5 Audit and Review Checklist

The system is periodically reviewed for:

- **Compliance:** Regular audits are conducted to verify compliance with performance, security, and governance standards.
- **Performance stability:** Monitoring logs and alert histories are reviewed to ensure timely responses to drift or failures and help ensure model stability.
- **Ethical adherence:** Ethical and regulatory considerations are reassessed periodically as data and usage contexts evolve.

Ethical and regulatory considerations are reassessed periodically as data and usage contexts evolve.

Reference Papers and Further Reading

- [1] Y. Koren, R. Bell and C. Volinsky, "Matrix Factorization Techniques for Recommender Systems," in *Computer*, vol. 42, no. 8, pp. 30-37, Aug. 2009, doi: 10.1109/MC.2009.263
- [2] S. Raza, M. Rahman, S. Kamawal, A. Toroghi, A. Raval, F. Navah, and A. Kazemeini, "A comprehensive review of recommender systems: Transitioning from theory to practice," arXiv:2407.13699, 2025. [Online]. Available: <https://arxiv.org/abs/2407.13699>
- [3] S. Rendle, C. Freudenthaler, Z. Gantner, and L. Schmidt-Thieme, "BPR: Bayesian personalized ranking from implicit feedback," arXiv:1205.2618, 2012. [Online]. Available: <https://arxiv.org/abs/1205.2618>
- [4] A. Milogradskii, O. Lashinin, A. P., M. Ananyeva, and S. Kolesnikov, "Revisiting BPR: A replicability study of a common recommender system baseline," in **Proc. 18th ACM Conf. Recommender Systems (RecSys '24)**, Oct. 2024, pp. 267–277, doi: 10.1145/3640457.3688073.
- [5] Y. Hu, Y. Koren and C. Volinsky, "Collaborative Filtering for Implicit Feedback Datasets," 2008 Eighth IEEE International Conference on Data Mining, Pisa, Italy, 2008, pp. 263-272, doi: 10.1109/ICDM.2008.22.
- [6] D. Xue, M. Li, C. Meng, and J. Zhang, "Core-elements subsampling for alternating least squares," arXiv:2509.18024, 2025. [Online]. Available: <https://arxiv.org/abs/2509.18024>
- [7] Y. Zhang, M. Zhang, Y. Liu, S. Ma, and S. Feng, "Localized matrix factorization for recommendation based on matrix block diagonal forms," in **Proc. 22nd Int. Conf. World Wide Web (WWW '13)**, Rio de Janeiro, Brazil, 2013, pp. 1511–1520, doi: 10.1145/2488388.2488520.
- [8] Hind I. Alshbanat, Hafida Benhidour, Said Kerrache, A survey of latent factor models in recommender systems, *Information Fusion*, Volume 117, 2025, 102905, ISSN 1566-2535, <https://doi.org/10.1016/j.inffus.2024.102905>.
- [9] E. Dervishaj and P. Cremonesi, "GAN-based matrix factorization for recommender systems," in **Proc. 37th ACM/SIGAPP Symp. Applied Computing (SAC '22)**, Apr. 2022, pp. 1373–1381, doi: 10.1145/3477314.3507099.

- [10] M. Winlaw, M. B. Hynes, A. Caterini, and H. De Sterck, “Algorithmic acceleration of parallel ALS for collaborative filtering: Speeding up distributed big data recommendation in Spark,” arXiv:1508.03110, 2016. [Online]. Available: <https://arxiv.org/abs/1508.03110>.
- [11] H. Steck, “Training and testing of recommender systems on data missing not at random,” in *Proc. 16th ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining (KDD ’10)*, Washington, DC, USA, 2010, pp. 713–722, doi: 10.1145/1835804.1835895.
- [12] Zhou, Yunhong & Wilkinson, Dennis & Schreiber, Robert & Pan, Rong. (2008). Large-Scale Parallel Collaborative Filtering for the Netflix Prize. 337-348. 10.1007/978-3-540-68880-8_32.
- [13] Takács, Gábor & Pilászy, István & Németh, Botyán & Tikk, Domonkos. (2008). Matrix factorization and neighbor based algorithms for the netflix prize problem. RecSys'08: Proceedings of the 2008 ACM Conference on Recommender Systems. 267-274. 10.1145/1454008.1454049.
- [14] Pan, Rong & Zhou, Yunhong & Cao, Bin & Liu, Nathan & Lukose, Rajan & Scholz, Martin & Yang, Qiang. (2008). One-Class Collaborative Filtering. 502-511. 10.1109/ICDM.2008.16.